Project Asteria - Detailed Notes **Topics Covered**

- Python classes and objects
- __init__ constructor and self
- Instance attributes vs parameters
- RobotConfig class design
- Robot state (battery, speed, movement, sensors)
- Methods and state updates
- Difference between methods and attributes
- Separation of responsibilities

1. RobotConfig Class

A class is a blueprint. An object is an instance created from that blueprint. RobotConfig stores configuration and current state of the robot.

Attributes implemented

robot_name, wheel_radius, max_speed, wheel_base, battery, is_moving, distance, temperature, braking.

Why __init__?

The constructor initializes every robot with a valid starting state. Attributes should exist from the moment the robot is created. For sensors, using None indicates 'no sensor reading yet'.

Methods built

show_config(): display current robot state.

change_speed(): updates speed and movement state.

stop_robot(): stops robot and updates movement state.

consume_power(): decreases battery and prevents invalid state.

update_distance(): stores latest distance sensor reading.

brakes(): checks distance and triggers stop logic.

Important Concepts

- Methods perform actions.
- Attributes store information.
- Do not assign values to methods (e.g. stop_robot = False).
- Call methods using parentheses: self.stop_robot().
- Do not call attributes as functions (is_moving()).
- Avoid duplicated logic by reusing methods.

Design Principles Learned

1. Separate updating sensor data from making decisions.
2. Represent unknown sensor readings with None.
3. Keep robot state internally consistent.
4. One method should ideally have one responsibility.

Current Asteria Progress

Completed: - Robot configuration - Speed control - Battery management - Motion state - Distance sensor - Initial braking logic Pending: - Direction control - Turning - Obstacle avoidance - Sensor fusion - Decision engine - Simulator

Next Coding Milestones

1. Direction state.
2. move_forward(), turn_left(), turn_right(), reverse().
3. Obstacle checking.
4. Grid-based simulator.
5. Physics simulator integration.